

AI Vendor, Procurement & Inventory Platform

Supporting detail behind the three-page overview

This document fills in what the overview deliberately left out: who uses the system, what each module holds, how work actually flows through it, what the database looks like, and exactly which job each layer of the AI does.

SCOPE, IN ONE LINE

One focused product for a small multi-outlet cloud kitchen — not a general-purpose ERP. Every design choice below follows from that narrowness.

1	Who uses it	page 2
2	Modules, and what gets built first	page 3
3	How work flows	page 4
4	Database	page 5
5	How the AI works	page 6
6	One recommendation, and the infrastructure	page 7

A NOTE ON THE DATA BEHIND THE DEMO

There is no live customer behind this design, so the demo runs on generated history. That history comes from a **separate simulator the application cannot see**: it holds the hidden truth about demand, vendor behaviour and price drift, and emits only the transaction records such a business would have left behind.

The forecast is then measured on a stretch of time it was never shown, and always against a simple baseline. If it cannot beat that baseline, the baseline is used and the accuracy screen says so. This is what keeps the intelligence honest rather than circular.

1 Who uses it

The target business is a cloud kitchen or small restaurant group running two to four outlets. Everything here is sized for that, and would need rethinking at ten times the scale.

WHAT THE SYSTEM HANDLES	TYPICAL VOLUME
Raw materials tracked	40 – 70
Vendors	8 – 15
Menu items with recipes	20 – 35
Purchase orders per week	15 – 30
Sales rows per outlet per day	20 – 35
History needed before the AI is useful	90 days minimum, 180 preferred

THREE ROLES, NOT FIVE

A three-outlet kitchen has an owner, a manager per outlet, and a part-time accountant. Modelling a separate procurement officer and inventory officer would be modelling an organisation chart that does not exist.

ROLE	SEES	CAN DO
Owner	All outlets	Everything — vendors, materials, recipes, approvals, costs and margins, analytics, AI advice, users, alert settings.
Outlet manager	Own outlet only	Raise a purchase order within a spend limit, receive goods, record wastage, run a stock count, import sales, request a transfer. Cannot see other outlets or record payments.
Accounts	All outlets, money only	View bills, record payments, mark a bill disputed, view spend reports. Cannot touch stock, orders or recipes.

VENDORS DO NOT LOG IN

In this segment vendors work on WhatsApp, not portals, and a login for them would double the security surface for no real gain. Instead the system produces a purchase-order PDF and a ready-to-send WhatsApp message that a person presses send on. A simple link-based confirmation page can come later.

2 Modules

MODULE	WHAT IT HOLDS	WHY IT HAS TO EXIST
Vendors	Contacts, rate per material with dates, payment terms, stated lead time	Rates are kept as history , so “this vendor raised atta 8% in two months” becomes a query instead of a memory .
Materials	Unit, pack size, category , shelf life	Reorder levels are deliberately <i>not</i> entered by the user — the system computes them . Asking the user hands the problem back.
Recipes	Material quantity per plate, plus a wastage allowance	The bridge that turns sales into consumption . Without it there is no AI.
Purchase orders	Lines, quantities, rates, expected date, status	Must support part delivery and short supply , because that is what really happens.
Inventory	Movement ledger, wastage, counts, transfers between outlets	The single record of what physically moved, and the basis of every stock figure.
Bills & payments	Bill, due date, part payments, ageing	Payments are <i>recorded</i> , never made. No gateway means no compliance burden and no insight lost.
AI insights	Recommendations with their reasons, plus the assistant	The reason the product exists rather than a spreadsheet.

The screens follow the modules: a dashboard, an AI insight inbox, the assistant, vendors, materials, recipes, purchase orders, stock, finance, CSV import, and a forecast-accuracy page. That last one is the screen that makes the rest believable.

What gets built first

VERSION 1 — BUILD NOW	LATER
Vendors with rate history , terms and lead time	Vendor login to confirm orders
Materials, recipes, two to three outlets	Central kitchen and bulk production
Purchase orders with part delivery and short supply	Multi-step approval workflows
Stock ledger, wastage, counts and transfers	Batch and expiry tracking per lot
Bills, due dates, part payments and ageing	Accounting software sync
CSV import for sales and opening stock	Direct POS and delivery-app integration
Reorder, deadstock, transfer and vendor advice	Measuring whether accepted advice helped
Forecast with an honest accuracy report	Reading vendor bill photos automatically

3 How work flows

THE PURCHASE LOOP — WHENEVER SOMETHING IS NEEDED

- 1 **A need appears** — either the AI recommends it, or a manager raises it directly.
- 2 **A purchase order is drafted** with a suggested vendor, a quantity rounded to pack size, and a by-when date.
- 3 **Approved** by the owner, or auto-approved if inside the manager's spend limit.
- 4 **Sent** as a PDF by email, and as a prefilled WhatsApp message a person presses send on.
- 5 **Goods arrive** — possibly partly, possibly with rejects. Each receipt is recorded separately against the order.
- 6 **Stock goes up** automatically, as new entries in the movement ledger.
- 7 **The bill is attached** and its due date is computed as delivery date plus the vendor's agreed credit days.
- 8 **Payment is recorded**, in full or in part, and the vendor's performance figures update on their own.

THE CONSUMPTION LOOP — EVERY DAY, AND THIS IS THE ENGINE

- 1 **Sales are imported** — dish, quantity, outlet, date — from a CSV or Excel export, or typed in.
- 2 **Recipes are applied** to convert dishes into raw materials used.
- 3 **Stock goes down** as consumption entries in the same ledger.
- 4 **Usage rate, cover and stockout risk** are recalculated per material, per outlet.

THE RECONCILE LOOP — WEEKLY, AND THIS IS THE TRUTH CHECK

- 1 **Someone counts the shelf** and enters the real quantity.
- 2 **The difference is recorded** against what the ledger expected — never by editing history, always as a new adjustment entry.
- 3 **That gap is the real insight.** Expected usage minus actual usage is wastage, over-portioning, theft, or a recipe that no longer matches the kitchen. Tracked per material, it is often worth more than the forecast.

THE NIGHTLY LOOP — UNATTENDED, AT 2AM

```
refresh analytics
  → run 14-day forecasts
  → recompute reorder points and safety stock
  → find deadstock and overstock
  → score vendors
  → check whether another outlet has spare stock
  → write tomorrow's recommendations, each with its reasons
  → raise alerts
```

4 Database

PostgreSQL, with an audit log alongside. Only the fields that matter are listed.

TABLE	IMPORTANT FIELDS
organizations	name, currency, timezone
locations	org, name — one row per outlet
users	org, name, email, role, and which locations they may access
vendors	org, name, phone, GSTIN, credit days, stated lead time
materials	org, name, unit, pack size, category, shelf life
vendor_materials	vendor, material, rate, minimum order, <i>valid from / valid to</i> — this is rate history
menu_items	org, name, selling price
recipe_components	menu item, material, quantity per plate, wastage % — a generic bill of materials
purchase_orders + items	outlet, vendor, status, expected date, who approved, which recommendation caused it; per line: material, quantity, rate
goods_receipts + items	order, outlet, received date; per line: accepted, rejected, reason
stock_movements	outlet, material, quantity change (+ or -), type, cost, what caused it, when
sales_records	outlet, date, menu item, quantity sold
stock_counts	outlet, date; per material: counted, expected, difference
invoices / payments	vendor, order, total, due date, amount paid, status; payments may be partial
recommendations	type, material, vendor, suggested quantity, by-when, rupee impact, reasons , input snapshot , status

THE MOST IMPORTANT DECISION IN THIS SCHEMA

There is **no current-stock column anywhere**. Stock on hand is always the sum of every movement for that material at that outlet, and movements are only ever added — a correction is a new entry, never an edit. Every inventory system that stores a quantity eventually drifts out of step with its own history. This one cannot, by construction.

HOW IT FITS TOGETHER, IN PLAIN WORDS

- A **vendor** supplies many **materials**, each at a rate valid between two dates — so price history is a fact, not a guess.
- A **menu item** is made of **materials** through its recipe. This is the link that turns sales into consumption.
- A **purchase order** has lines; receiving it creates one or more **receipts**, which is how part delivery works naturally. Receipts, sales and counts all add entries to the ledger.

5 How the AI works

Most of what is sold as “AI” in this space is arithmetic. Being clear about which part is which is what makes the system trustworthy.

LAYER 1

Plain code — no model

Stock balances, order totals, bill due dates, outstanding amounts, ageing buckets, order status. Exact by requirement. A model here would be a bug.

LAYER 2

Analytics — formulas, not machine learning

Average daily usage and how erratic it is. Real vendor lead time, taken at the **85th percentile** rather than the average, because you plan for the bad week. Delay rate, fill rate, reject rate, price drift. Days of cover and days to stockout. Deadstock is simply no outward movement for N days while stock remains; overstock is cover far beyond what shelf life or lead time justifies.

Reorder point = average daily usage × lead time at the 85th percentile + safety stock, where safety stock grows with demand variability. This is standard inventory theory, and calling it a formula rather than AI is a strength: it can be audited line by line.

LAYER 3

Forecast — the only genuine model

Fourteen-day demand per material, per outlet. Candidates in order of cost: same weekday last week as the **baseline**, a weekday-weighted moving average, a method built for items used only twice a week, and gradient boosting on calendar features. Each is tested on time it has never seen and the winner is chosen per material. **If nothing beats the baseline, the baseline is used and the accuracy screen says so.**

The forecast returns a range, not just a number, and that uncertainty feeds straight into safety stock — so a material the system understands poorly automatically gets a bigger buffer.

LAYER 4

Claude — explains and answers, never calculates

Explaining: it receives the already-computed numbers and writes one sentence. Every figure in its output is checked against the figures it was given; if a number appears that was not supplied, the system falls back to a fixed template.

Answering: typed questions are handled by calling a fixed set of read-only functions — low stock, deadstock, pending payments, vendor performance, spend, price history, explain a recommendation. Claude picks which to call; the backend runs the query. It never writes SQL, which keeps one outlet’s data out of another’s answers and makes wrong-but-plausible totals impossible.

6 One recommendation, end to end

computed by layers 2 and 3:

paneer, Indiranagar	on hand 4.2 kg	used 6.8 kg/day
runs out in	0.6 days	
vendor lead time (85th percentile)	2 days	
suggested	22 kg from Sri Dairy	
alternative	Milk Mart, 6% cheaper, late on 38% of orders	
value at risk	Rs 6,100	

WHAT CLAUDE TURNS THAT INTO

“Paneer at Indiranagar will run out tomorrow — 4.2 kg left against 6.8 kg used per day. Order 22 kg from Sri Dairy today; their lead time runs to 2 days at worst. Milk Mart is 6% cheaper but late on 38% of orders, which is why it is not the pick.”

Note what happened: the *decision* and every number came from layers 2 and 3. Claude only chose the words. The vendor was not picked for being cheapest either — rate, reliability, fill rate, quality and credit terms are combined into a weighted score whose contributions are shown on screen. If the owner accepts the card, it becomes a purchase order that remembers which recommendation created it, so the advice can be judged later.

7 Infrastructure

CONCERN	SERVICE	TIER
Frontend hosting	Vercel	Free
Backend hosting	Render	Free
Database and login	Supabase (Postgres + Auth)	Free
Bill photos, PO PDFs	Supabase Storage	Free
Nightly and weekly jobs	Render Cron	Free
Email	Resend	Free
Error tracking	Sentry	Free
Language model	Claude API	Pay per use, small

Two practical notes. Supabase is used as managed Postgres, login and file storage only — the business logic stays in the Python backend, because the analytics and forecasting layers need to live together. And the free backend tier sleeps when idle, so a scheduled ping keeps it warm; without it, the first click of a live demo waits half a minute.